ENEE 467: Robotics Project Laboratory
Dynamic Programming and Tabular Reinforcement Learning
Lab 10

Learning Objectives

The purpose of this lab is to provide a first hands-on experience with reinforcement learning using the Gymnasium framework. After completing this lab, students should be able to:

1. Implement model-based RL algorithms: value iteration and policy iteration, to understand the RL loop, convergence, and exploration-exploitation tradeoffs.
2. Implement model-free RL algorithms: Q-Learning and SARSA, to understand temporal-differencing, tune hyperparameters and compare performances of various RL models.

Together, these exercises build the conceptual and practical foundation for deep RL in continuous-space environments and deep RL.

Related Reading

- R. S. Sutton and A. G. Barto, "Reinforcement Learning: An Introduction" (Second Edition), MIT Press, 2018. Open Access

1 Lab Instructions

To prepare your system for the labs, follow these steps to install the necessary software. We will use a command-line interface (CLI) for all installations. Your lab machine should be running Ubuntu 20.04 or a newer version. This ensures compatibility with the required libraries and drivers. If you're not on a compatible system, consider using a virtual machine (e.g., VirtualBox, VMware, WSL2) or the UMIACS computing service.

1.1 Software Installation

It is recommended to use a `conda` environment that can be installed using Miniconda. Then, create a `conda` environment and activate it:

```
conda create --name rl467 python=3.11
conda activate rl467
```

Now install the Gymnasium library for RL tasks.

```
pip install swig
pip install gymnasium[box2d]
```

Test your installation:

```python
import gymnasium as gym

# Initialise the environment
env = gym.make("LunarLander-v3", render_mode="human")

# Reset the environment to generate the first observation
observation, info = env.reset(seed=42)
for _ in range(1000):
    # this is where you would insert your policy
    action = env.action_space.sample()

    # step (transition) through the environment with the action
    # receiving the next observation, reward
    # and if the episode has terminated or truncated
    observation, reward, terminated, truncated, info = env.step(action)

    # If the episode has ended then we can reset to start a new episode
    if terminated or truncated:
        observation, info = env.reset()

env.close()
```

> Known issue with rendering using integrated graphics: 'libGL error: MESA-LOADER: failed to open iris'. A solution is to run the following command in the conda environment:
>
> ```
> conda install -c conda-forge libstdcxx-ng
> ```

1.2 Implementation: Random Policy

To familiarize yourself with the Gymnasium-style API, the above code subsection 1.1 implements a random policy that uniformly samples actions in the `LunarLander` environment. Modify the code to reflect the FrozenLake environment, and visualize the random policy. You can also experiment with other environments in Gymnasium.

1.3 Implementation: Tabular Model-based RL

You will implement tabular model-based RL algorithms—value iteration and policy iteration—for a grid-world setting using dynamic programming. The provided file `dynamic_programming.py` contains the skeletal code for the FrozenLake environment where you will implement the algorithms. The tasks to be completed are as follows:

1. Complete the code for both value iteration and policy iteration. (Refer to the reading material for pseudo-code).
2. Provide visualizations of the learned policy.

3. Change the FrozenLake environment to have stochastic transitions: set the parameter `is_slippery=True`.
4. Modify the reward function by,
   (a) adding a positive scalar value to the reward at each step, e.g., $r' = r + 1$.
   (b) multiplying the reward at each step by a scalar value, e.g., $r' = r * 2$.
   (c) adding a penalty for each step, e.g., $r' = r - 0.1$.

Questions

1. Compare how the value function and policy change for each reward modification.
2. Does the optimal policy change? Why or why not?
3. How does each reward modification affect the convergence of each algorithm?
4. How do deterministic vs. stochastic environments (i.e., setting `is_slippery=True`) influence algorithm performance?

1.4 Implementation: Tabular Model-free RL

You will implement tabular model-free RL algorithms—Q-Learning (off-policy) and SARSA (on-policy)—for a grid-world setting. The provided file `tabular_rl.py` contains the skeletal code for the FrozenLake environment where you will implement the algorithms, plot the results and tune the hyperparameters. The tasks to be completed are as follows:

1. Fill in the code for updating the Q-function in the training loop of `q_learning`.
2. Similar to the style of `q_learning`, create a new function `sarsa`, which has the modified Q update according to the SARSA update rule.
3. Plot the evolution of rewards over timesteps or episodes, and compare the performance (plots) of the algorithms. Change the environment `seed` and plot the results. Are the results same across different `seed` values?
4. Change the values of the hyperparameters and show the corresponding rewards–vs–time plots.
5. The current code implements a constant ϵ-greedy action selection strategy. Modify this with your own ϵ-decay schedule (e.g., linear decay, exponential decay, step decay, etc.). Plot the results of rewards–vs–time for your choice(s).
6. Provide visualizations of the learned policy.
7. Change the FrozenLake environment to have stochastic transitions by setting `is_slippery=True`.
8. Repeat the steps for Taxi and CliffWalking environment, by changing the environment IDs.

Questions

1. Which of the algorithms has a stable learning curve?
2. How do the hyperparameters affect the performances of the policies?
3. How does stochasticity affect the performances of the policies?
4. In the algorithms implemented so far, we explicitly represent a table for the Q-function and value function. What limitations might this approach face in larger or continuous environments?